

软件安全实验三

班级：2022211801 学号：2022211570 姓名：项枫

一、目标

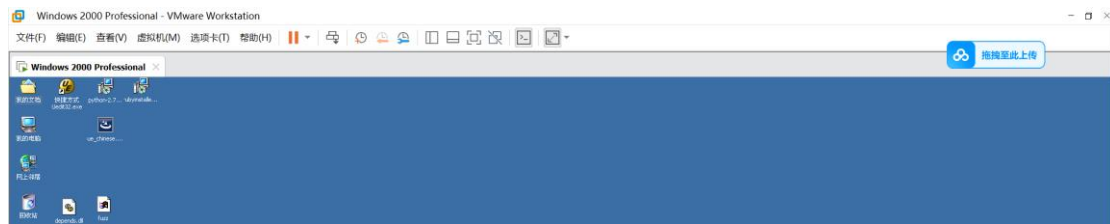
- 1、了解 shellcode 注入原理
- 2、理解给出的弹出对话框的汇编代码
- 3、通过淹没静态地址来实现 shellcode 的代码植入
- 4、通过跳板来实现 shellcode 的代码植入
- 5、尝试修改汇编语句的 shellcode 实现修改标题等简单操作

二、测试步骤与结果

1、创建可执行程序

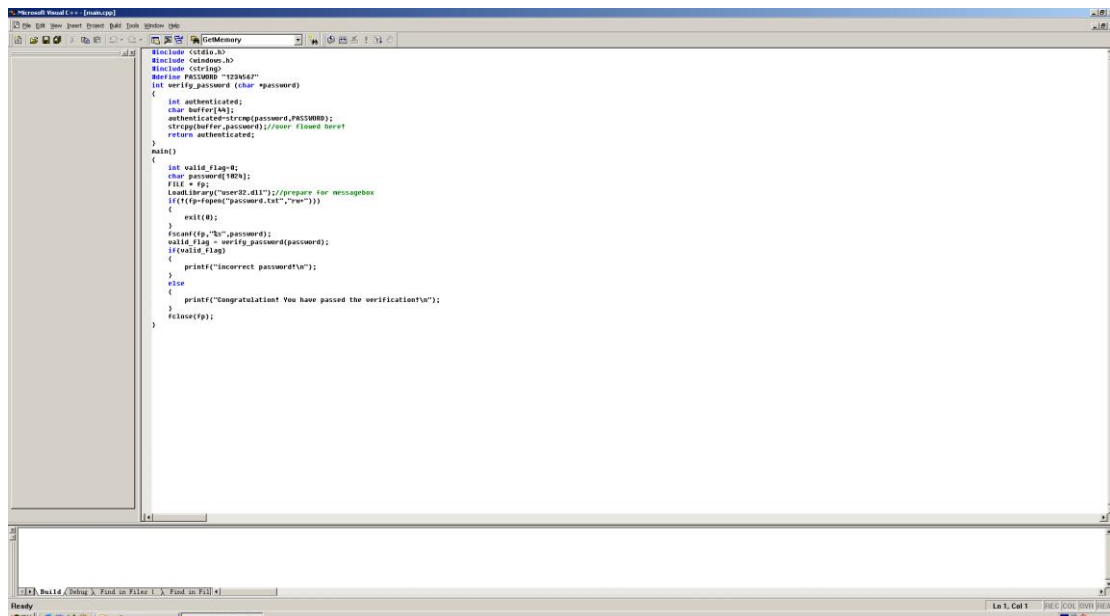
(1) 进入虚拟机

打开 VM workstation，点击文件->打开->选择我们要使用的 WIN2000 的 VMX 文件，之后我们就进入了虚拟机。

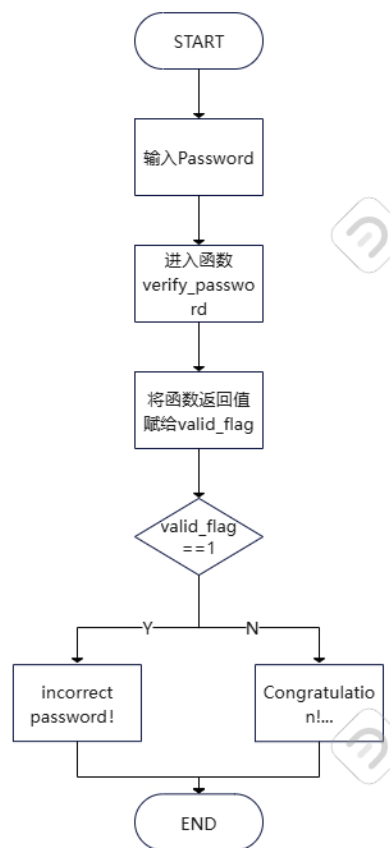


(2) 研究代码

使用桌面的 VC6 打开文件夹“lab3\overflow_exe”中的 main.cpp，对代码进行研究。



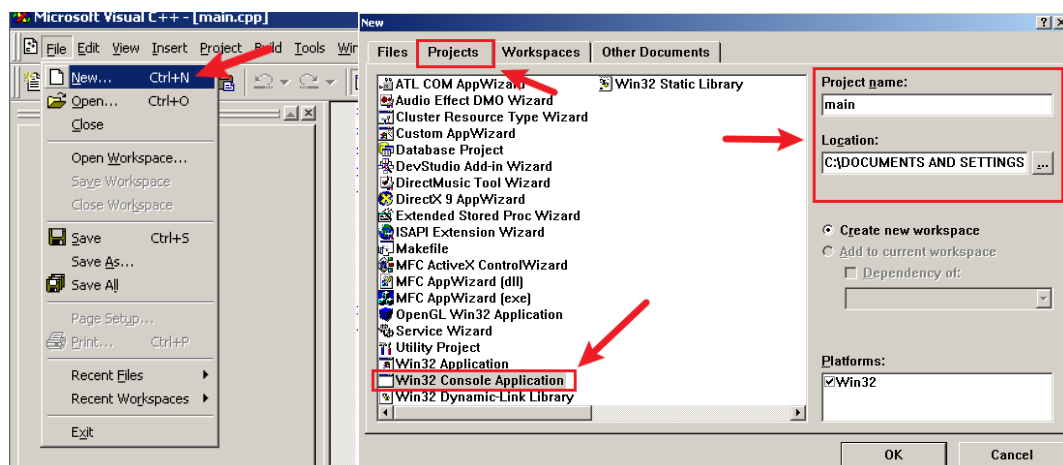
显而易见，这是一个验证输入密码是否正确的简单小程序，代码逻辑流程图如下：



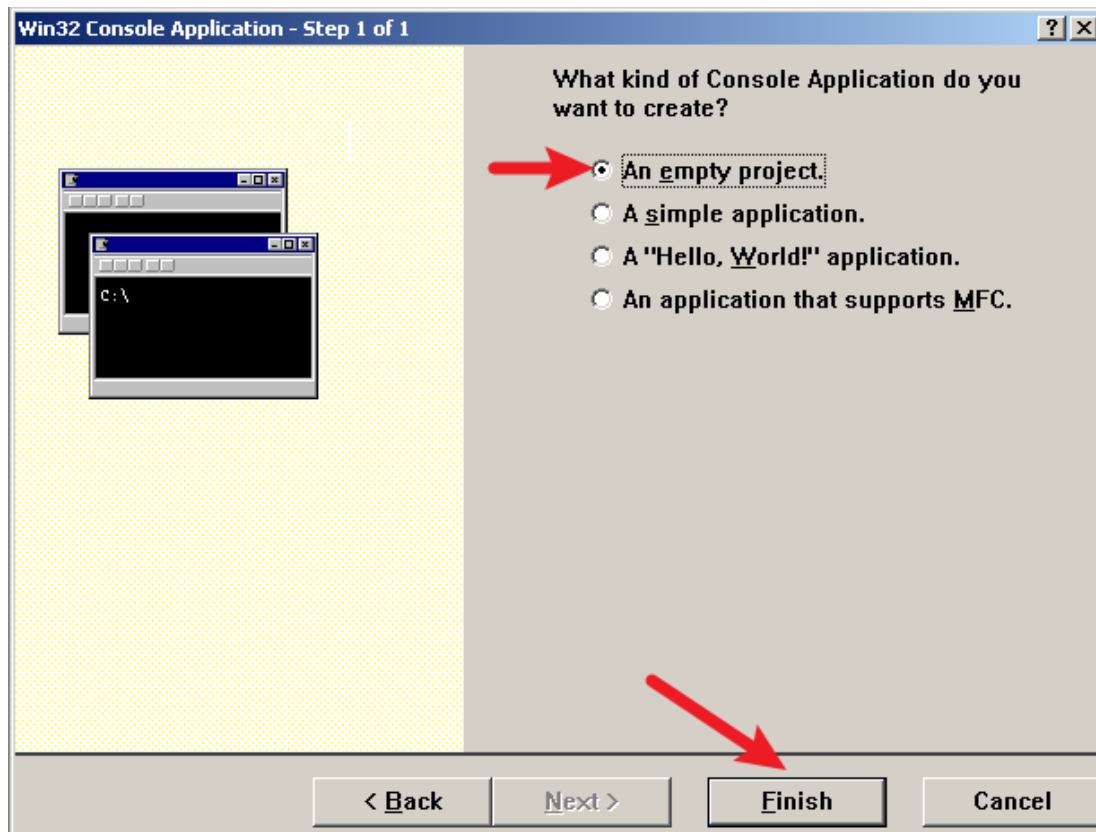
这个程序与之前的实验的程序有 3 处不同：1) 增加头文件 windows.h，以便能调用 LoadLibrary 函数去装载 user32.dll；2) Verify_password 的 buffer 变量增加到 44 字节，以便能够承载我们的 shellcode；3) Main 函数装载 user32.dll 以便能在植入代码中调用 MessageBox。

(3) 生成工程

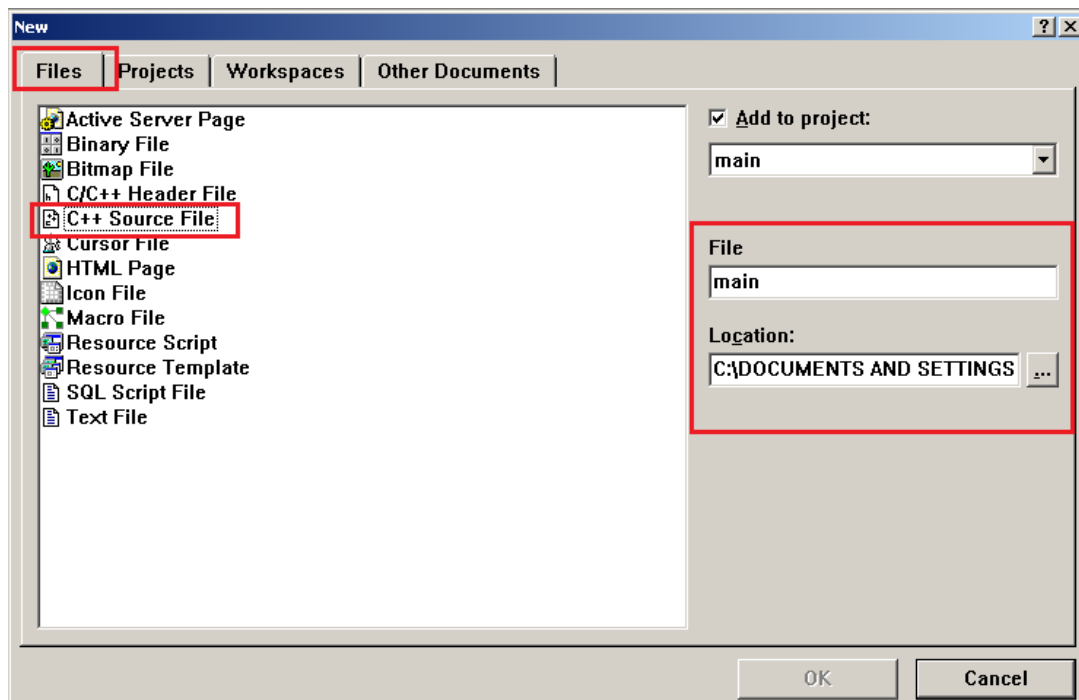
①在 VC 中，点击 File->new->Projects->Win32 Console Application，右上角设置好 project 的名称和存放路径。



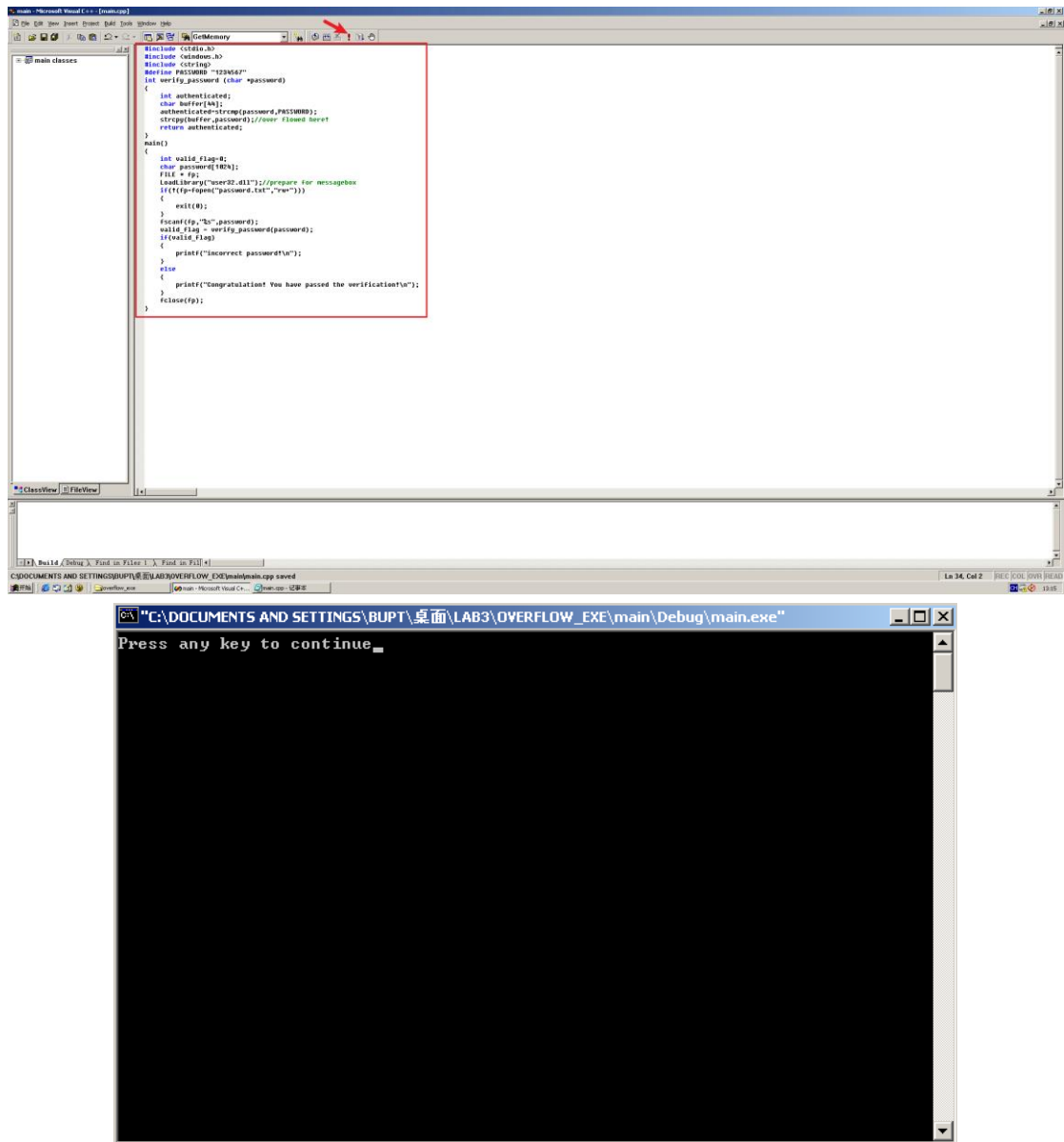
② 点击 OK->An empty project->Finish。



③之后再 new->Files->C++ Source File, 右侧设定好添加到的项目(我们刚才新建的项目), 文件名和路径。

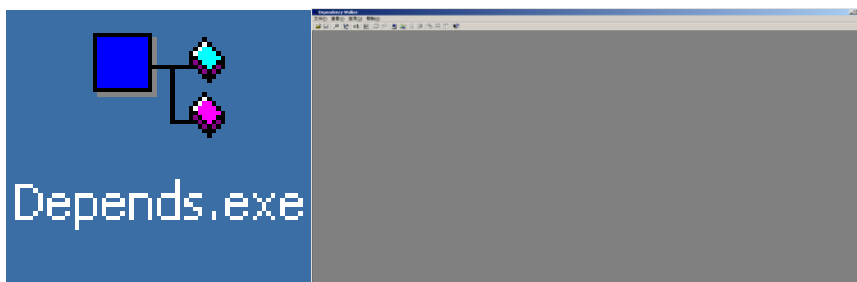


④完成新建文件后，在新建的文件下编写或复制刚才我们看到的 main 程序，然后点击右上的红色感叹号执行。

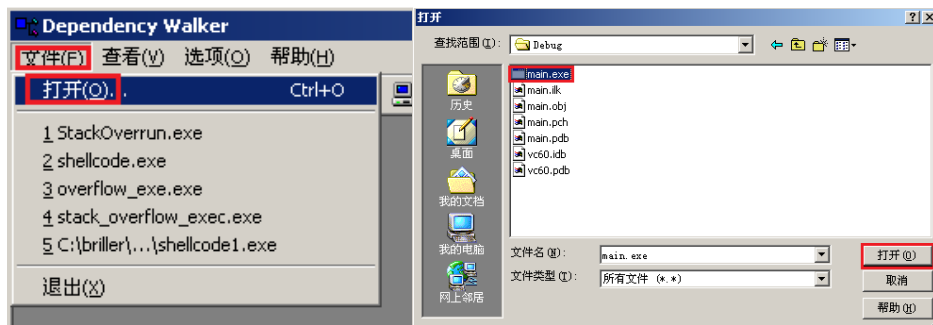


2、寻找 dll 中函数地址

(1) 点击桌面上的 depends.exe 进入程序。



(2) 打开我们生成的 exe 文件。



(3) 点击剖析->开始剖析，参数保持默认即可。



3、查找并计算 ExitProcess 函数的入口地址

(1) 点击 kernel32.dll，在右侧能找到 ExitProcess 的函数入口点 0x0001B0BB。

E	序号 ^	提示	函数	入口点
	140 (0x008C)	139 (0x008B)	EscapeCommFunction	0x0003C44F
	141 (0x008D)	140 (0x008C)	ExitProcess	0x0001B0BB
	142 (0x008E)	141 (0x008D)	ExitThread	0x000106CF
	143 (0x008F)	142 (0x008E)	ExitVDM	0x00002689
	144 (0x0090)	143 (0x008F)	ExpandEnvironmentStringsA	0x00014CA6
	145 (0x0091)	144 (0x0090)	ExpandEnvironmentStringsW	0x00008178
	146 (0x0092)	145 (0x0091)	ExpungeConsoleCommandHistoryA	0x00044A58
	147 (0x0093)	146 (0x0092)	ExpungeConsoleCommandHistoryW	0x00044A4A
	148 (0x0094)	147 (0x0093)	ExtendVirtualBuffer	0x00034283
	149 (0x0095)	148 (0x0094)	FatalAppExitA	0x00037AFB

(2) 在下面的窗口中能够查到 kernel32.dll 的实际基址 0x77E60000。

模块	文件的日期	连接的时间	文件大小	属性	连接的时间	校验和	CPU	子系统	序号	版本基址	实际基址	模块大小	数据偏移	文件版本	产品版本	语言版本	连接的时间	OS版本	子系统版本
KERNEL32.DLL	2000-01-10 12:00	1999-12-11 1:14	860,944	A	0x00000000	0x00000000	x86	Console	0	0x77E60000	0x77E60000	0x00004000	3	5.0.2191.1	5.0.2191.1	5.0	5.12	5.0	4.0
MAIN.EXE	2000-01-10 12:00	1999-12-11 1:14	102,864	A	0x00000000	0x00000000	x86	Console	0	0x00000000	0x00000000	0x00001000	1	5.0.2191.1	5.0.2191.1	5.0	5.12	5.0	4.0

(3) 将两个地址相加即可得到 ExitProcess 的入口地址：

$$0x0001B0BB + 0x77E60000 = 0x77E7B0BB$$

4、查找并计算 MessageBoxA 函数的入口地址

(1) 点击 user32.dll，在右侧能找到 MessageBoxA 的函数入口点 0x00033D68。

E	序号 ^	提示	函数	入口点
	449 (0x01C1)	448 (0x01C0)	MenuWindowProcA	0x0003B535
	450 (0x01C2)	449 (0x01C1)	MenuWindowProcW	0x0003B4E5
	451 (0x01C3)	450 (0x01C2)	MessageBeep	0x00013600
	452 (0x01C4)	451 (0x01C3)	MessageBoxA	0x00033D68
	453 (0x01C5)	452 (0x01C4)	MessageBoxExA	0x000341DB
	454 (0x01C6)	453 (0x01C5)	MessageBoxExW	0x000216F4
	455 (0x01C7)	454 (0x01C6)	MessageBoxIndirectA	0x000484FF
	456 (0x01C8)	455 (0x01C7)	MessageBoxIndirectW	0x00027328
	457 (0x01C9)	456 (0x01C8)	MessageBoxW	0x000216CC
	458 (0x01CA)	457 (0x01C9)	ModifyMenuA	0x00021C9E

(2) 在下面的窗口中能够查到 user32.dll 的实际基址 0x77DF0000。

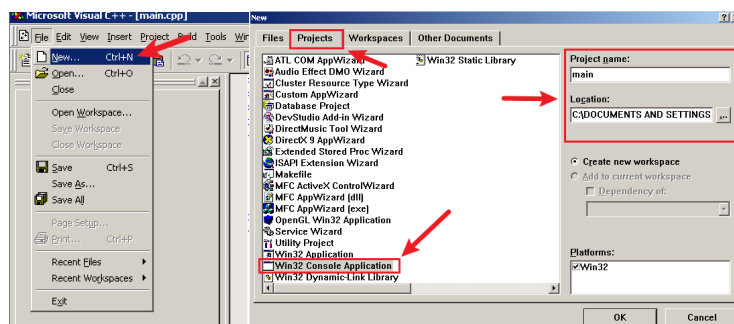
模块	文件时间戳	连接时间戳	文件大小	属性	连接依赖项	依赖项	CPU	子系统	序号	优先基址	实际基址	虚拟大小	数据源	文件版本	产品版本	编译版本	连接版本	OS版本	子系统版本
kernel32.dll	2000-01-10 12:00	1999-12-11 1:14	860,944	A	0x0000CF37	0x0000CF37	x86	Console	DBG	0x77E40000	0x77E40000	0x00000000	3	5.0.2191.1	5.0.2191.1	5.0	5.12	5.0	4.0
main.exe	2024-10-24 13:17	2024-10-24 13:17	192,402	A	0x00000000	0x00000000	x86	Console	PE8	0x00400000	0x00400000	0x00001000	1	N/A	N/A	0.0	6.0	4.0	4.0
ntdll.dll	2000-01-10 12:00	1999-10-28 4:06	481,040	A	0x0007E32E	0x0007E32E	x86	Console	DBG	0x77F80000	0x77F80000	0x00079000	2	5.0.2163.1	5.0.2163.1	5.0	5.12	5.0	4.0
ADVAPI32.dll	2000-01-10 12:00	1999-12-11 1:14	357,648	A	0x0006384A	0x0006384A	x86	Console	DBG	0x77D90000	0x77D90000	0x0005A000	7	5.0.2191.1	5.0.2191.1	5.0	5.12	5.0	4.0
GDI32.dll	2000-01-10 12:00	1999-11-12 16:44	229,256	A	0x0009401C	0x0009401C	x86	Console	DBG	0x77F40000	0x77F40000	0x0003C000	5	5.0.2180.1	5.0.2180.1	5.0	5.12	5.0	4.10
RPCRT4.dll	2000-01-10 12:00	1999-12-11 1:15	97,040	A	0x000180F7	0x000180F7	x86	GUI	DBG	0x75E00000	0x75E00000	0x0001A000	6	5.0.2180.1	5.0.2180.1	5.0	5.12	5.0	4.0
USER32.dll	2000-01-10 12:00	1999-12-11 1:14	444,688	A	0x00070016	0x00070016	x86	Console	DBG	0x77D00000	0x77D00000	0x0006F000	8	5.0.2193.1	5.0.2193.1	5.0	5.12	5.0	4.10
USER32.dll	2000-01-10 12:00	1999-12-11 1:14	400,144	A	0x0006AF3F	0x0006AF3F	x86	GUI	DBG	0x77DF0000	0x77DF0000	0x00064000	14	5.0.2180.1	5.0.2180.1	5.0	5.12	5.0	4.0

(3) 将两个地址相加即可得到 MessageBoxA 的入口地址：

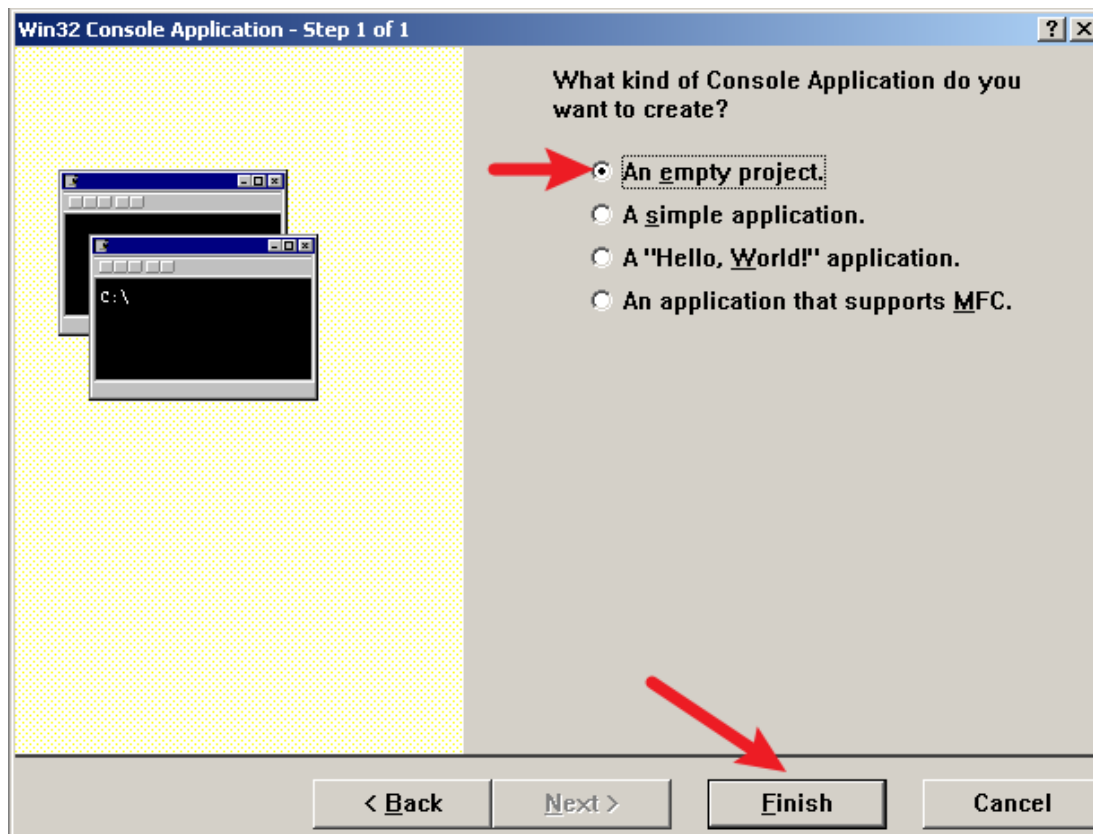
$$0x00033D68 + 0x77DF0000 = 0x77E23D68$$

5、调试 shellcode

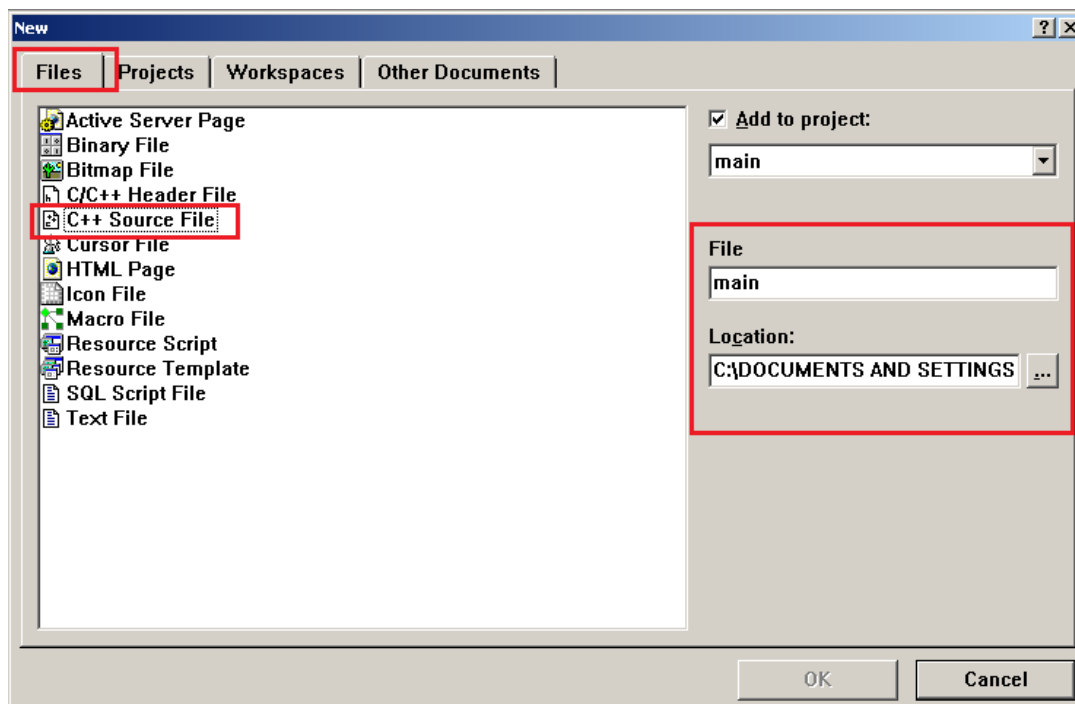
(1) 在 VC 中，点击 File->new->Projects->Win32 Console Application，右上角设置好 project 的名称和存放路径。



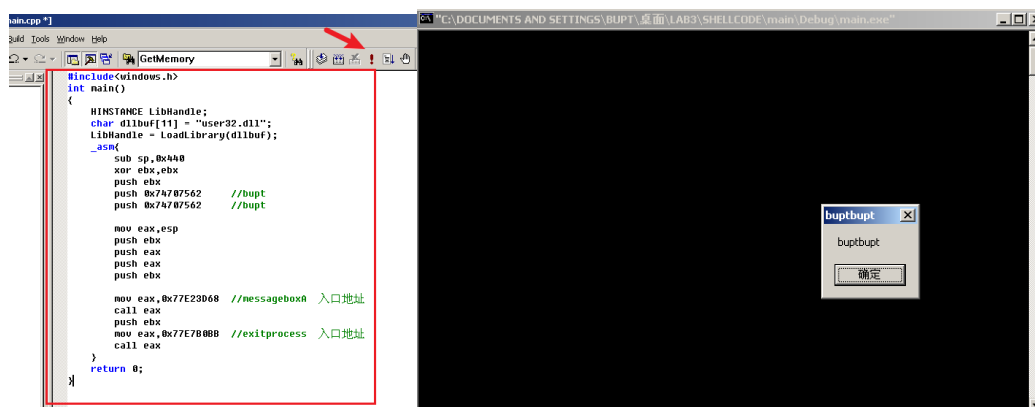
(2) 点击 OK->An empty project->Finish。



(3) 之后再 new->Files->C++ Source File, 右侧设定好添加到的项目(我们刚才新建的项目), 文件名和路径。

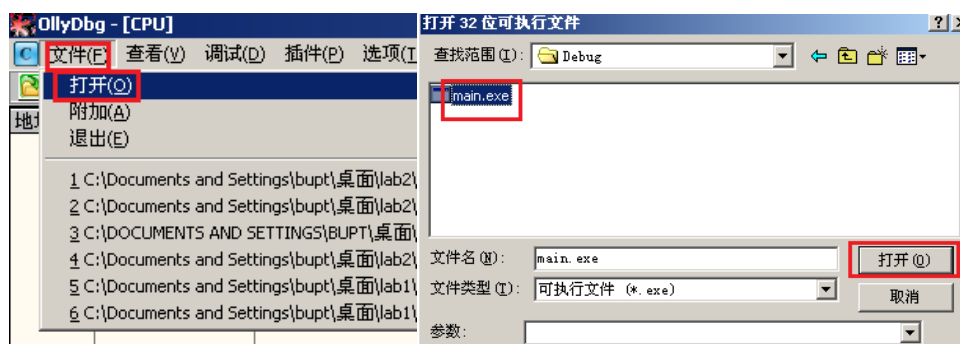


(4) 建文件后, 在新建的文件下编写或复制刚才我们看到的 main 程序, 两个函数的入口必须修改成刚才所计算出的地址, 然后点击右上的红色感叹号执行, 成功弹出对话框。

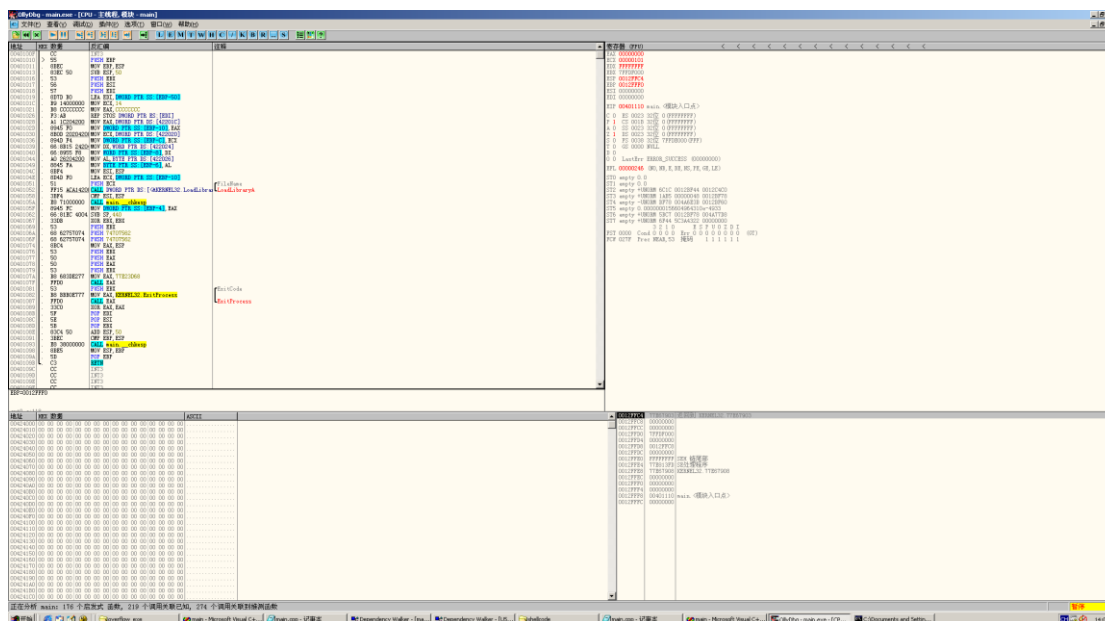


6、获取 shellcode 的指令码

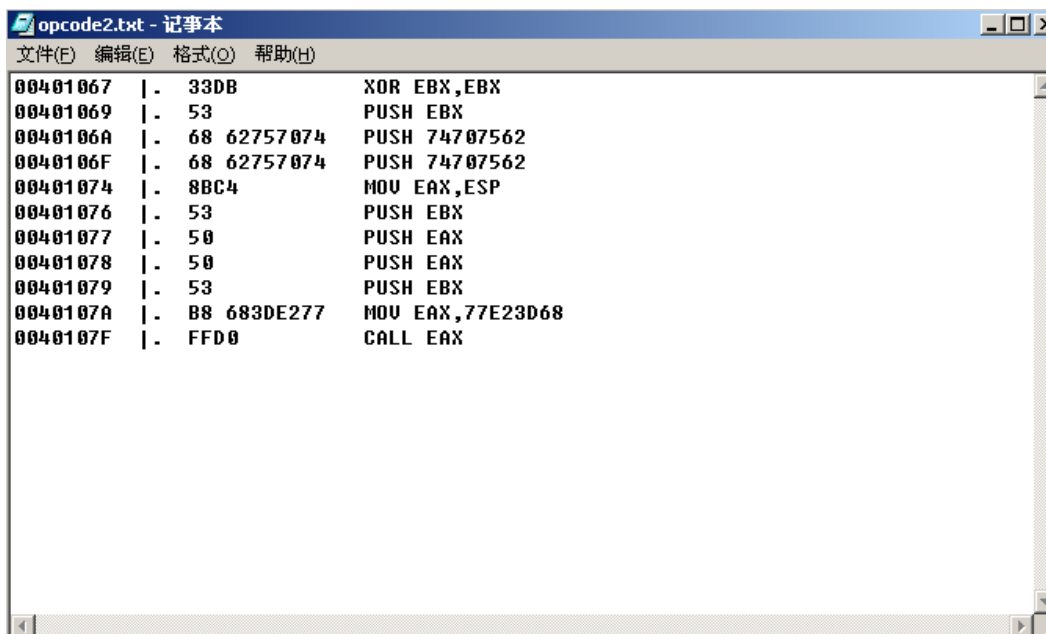
(1) 我们用 ollydbg 打开 “shellcode\main\debug\main.exe” 文件。



(2) 找到编写 shellcode 的汇编语言。

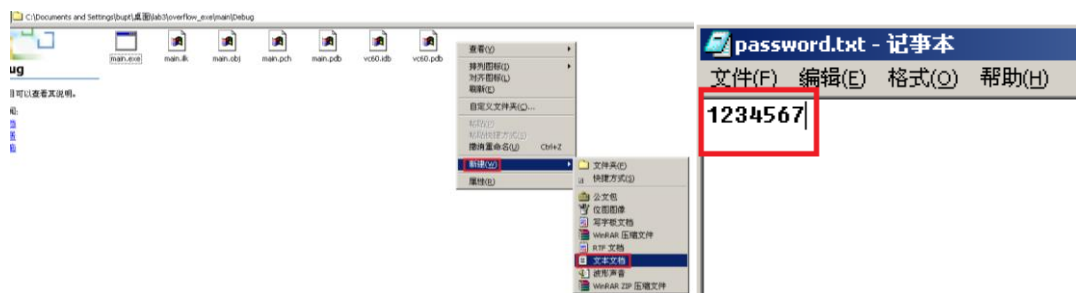


(3) 复制选定我们需要的部分：XOR EBX,EBX 开始，CALL EAX 结束，右键复制到文件，然后保存，我们就得到了操作码的文件。

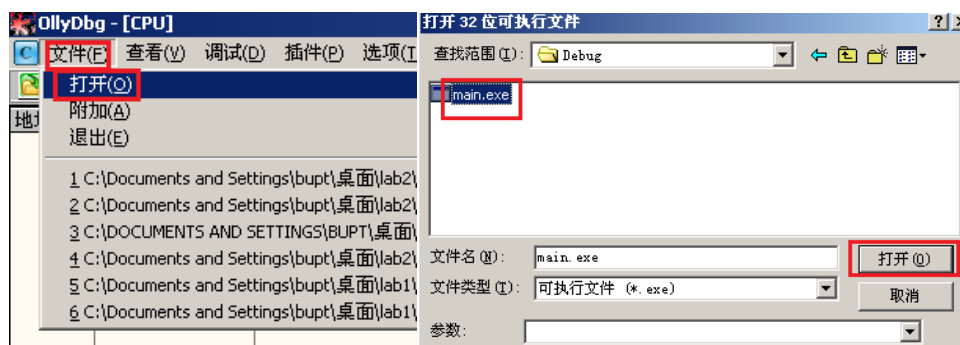


7、查找 overflow_exe 中的 shellcode 的起始地址

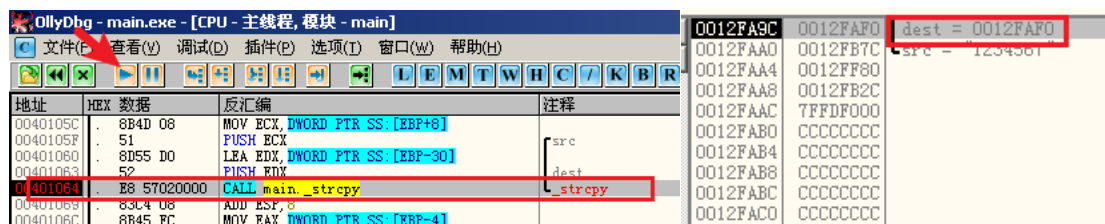
(1) 在“overflow_exe\main\debug”文件夹下创建 password.txt, 并在其中写入 1234567。



(2) 我们用 ollydbg 打开 “overflow_exe\main\debug\main.exe” 文件。

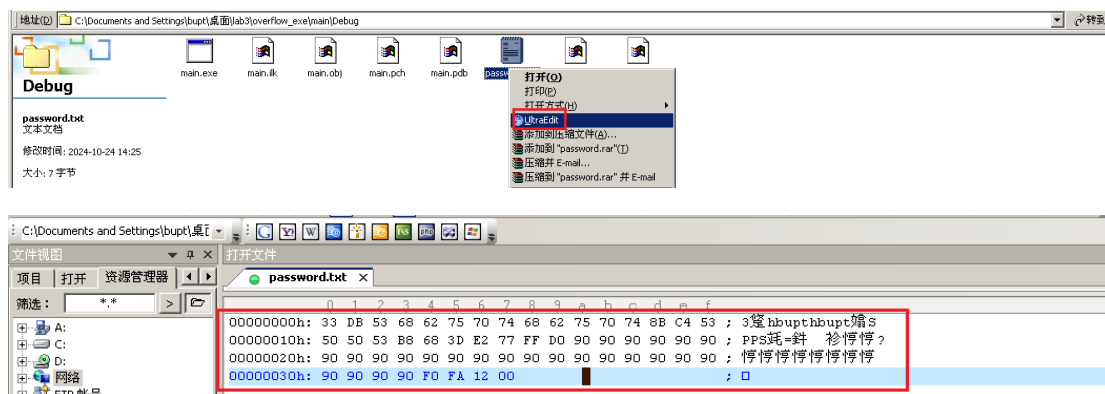


(3) 在 strcpy 处添加断点 (F2)，开始执行到断点处。查看缓冲区信息，可以看到 dest 指向地址 0x0012FAF0，就是我们要注入的 shellcode 的起始地址。

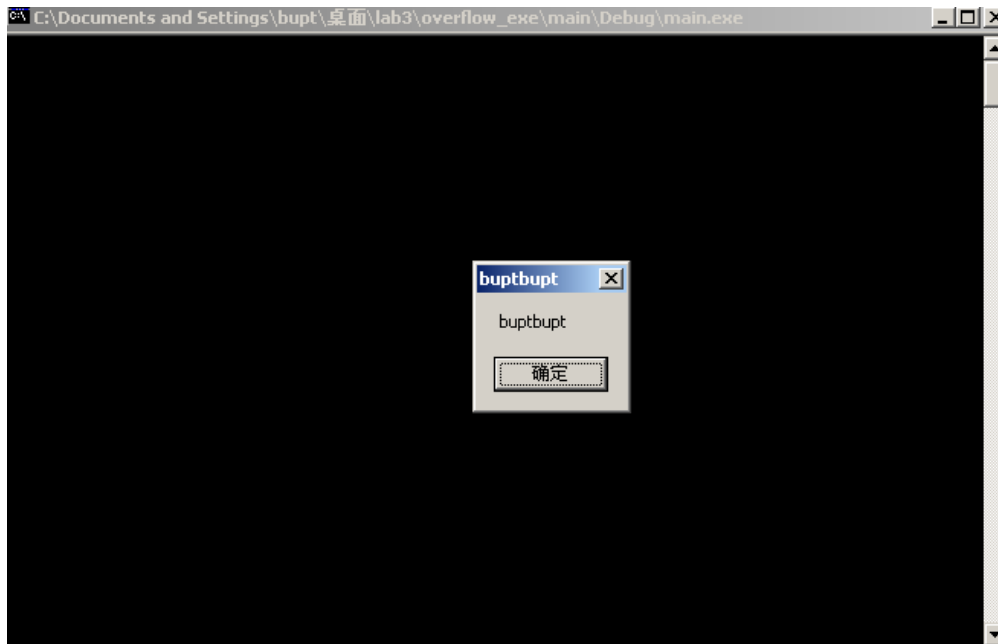


8、采用静态地址跳转的方法进行注入

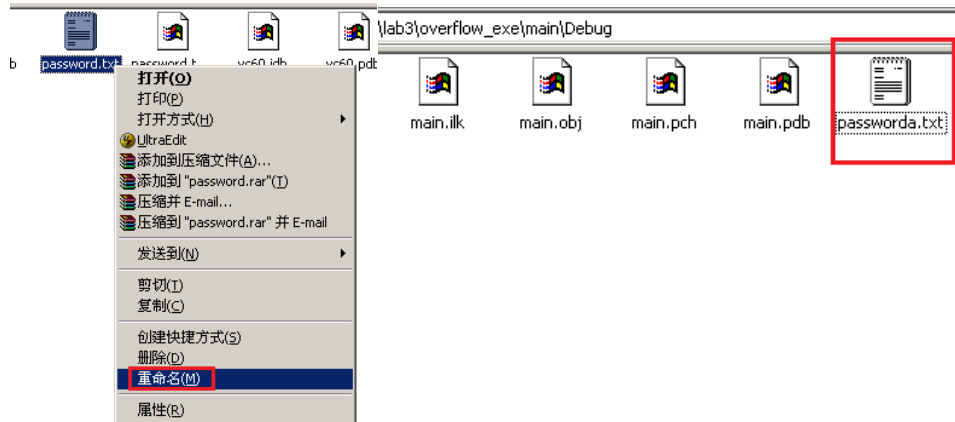
(1) 用 UltraEdit 打开 password.txt，并切换成 16 进制编辑的方式。构造的 txt 文件格式应该是 Shellcode + 0x90 若干 + shellcode 在缓冲区的起始地址（注意逆序填写），通过调整 0x90 的数量来保证第 53-56 字节是 shellcode 在缓冲区的起始地址。



(2) 重新运行 “overflow_exe\main\debug\overflow.exe”，可以看到弹出窗口，说明通过淹没静态地址成功实现了 shellcode 的代码植入。

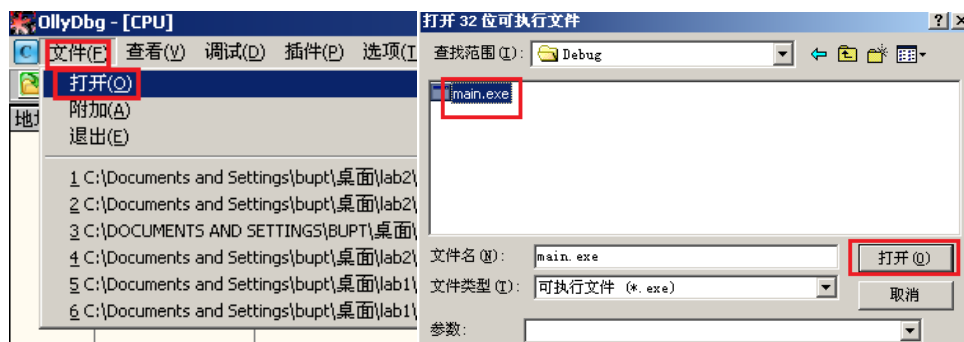


(3) 将 password.txt 重命名为 passworda.txt，因为之后还要新制作一个名为 password 的 txt 文件。



9、利用跳板来进行 shellcode 注入

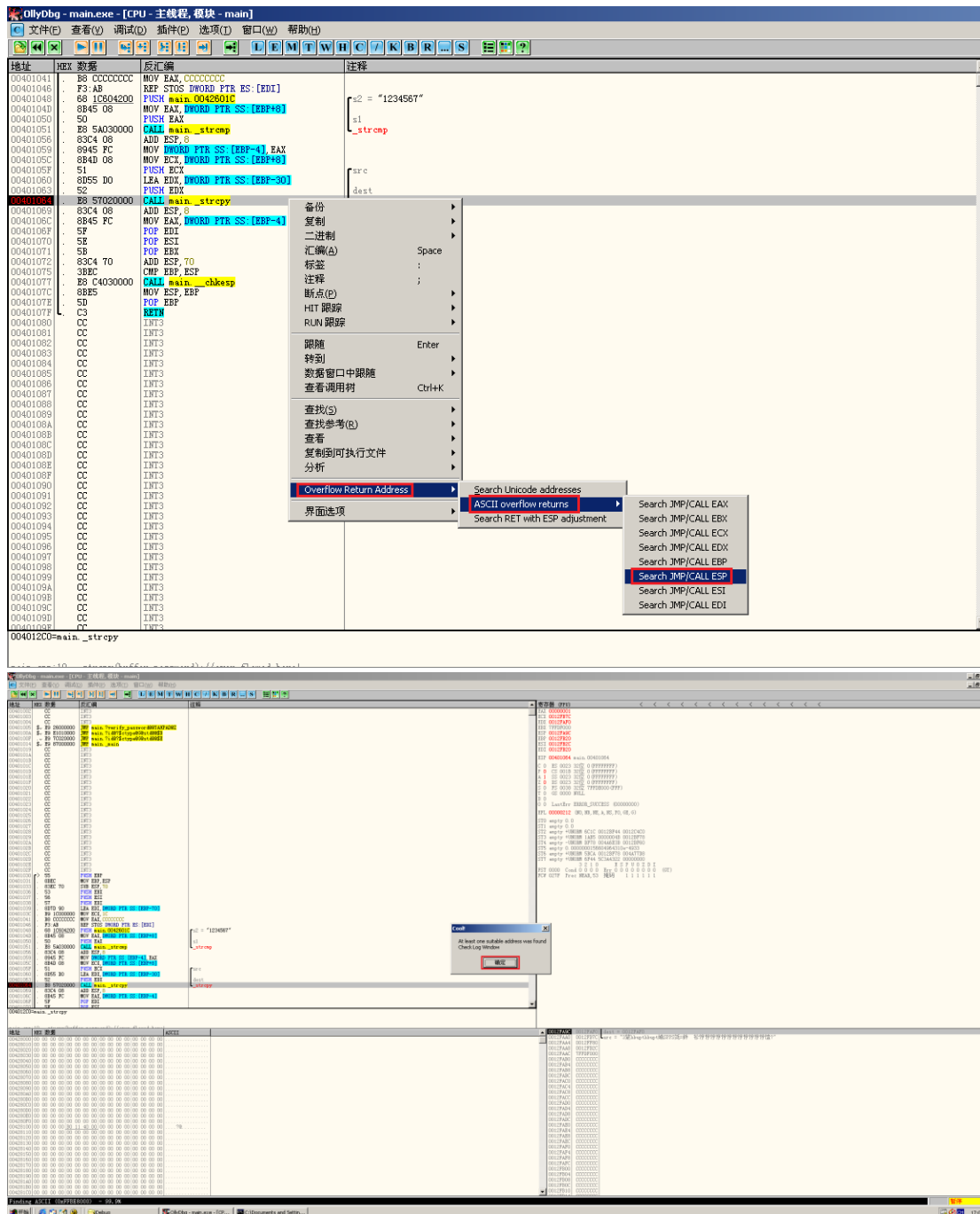
(1) 我们用 ollydbg 打开 “overflow_exe\main\debug\overflow.exe”。



(2) 在 strcpy 函数上设置断点，运行至断点。



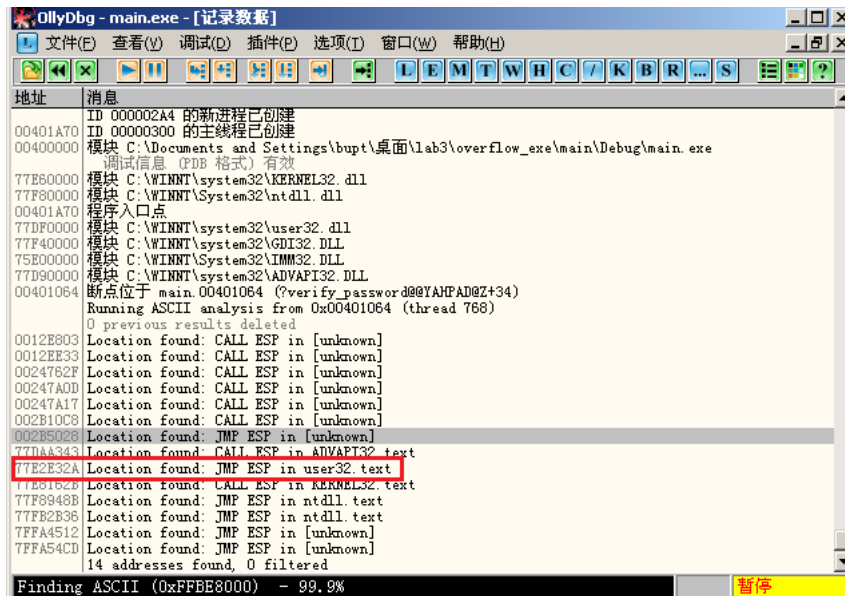
(3) 右键选择 overflow return address -> ASCII overflow returns -> search JMP/CALL ESP。



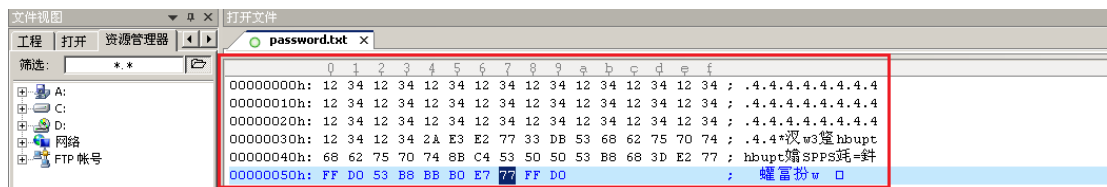
(4) 搜索完毕后点击日志查看(L 图标)的按钮。



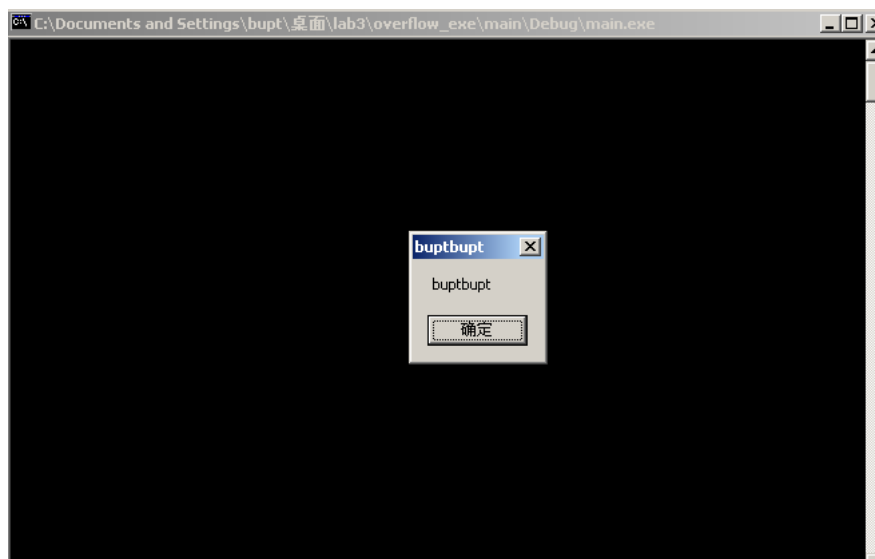
(5) 可以看到一条在 user32.text 中的 JMP ESP 指令的地址为 0x77E2E32A。



(6) 用 ultraedit 打开 password.txt 这次的文件结构为：52 字节填充物 + 4 字节 JMP ESP 地址（逆序）+ shellcode（可选 + 若干 0x90）。

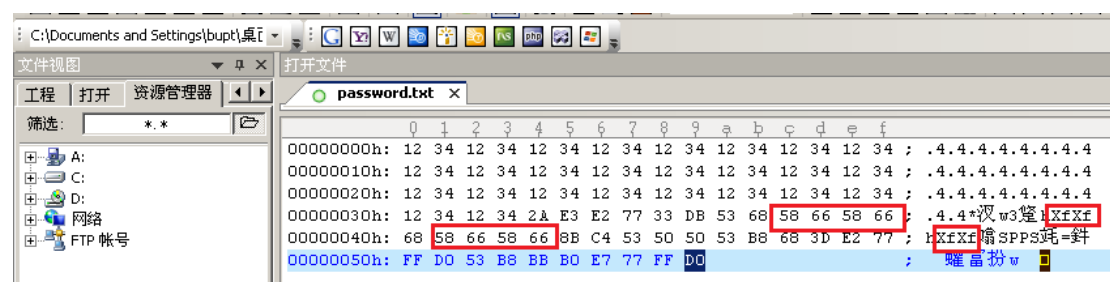


(7) 重新运行 “overflow_exe\main\debug\main.exe”，可以看到弹出窗口，说明通过跳板成功实现了 shellcode 的代码植入。

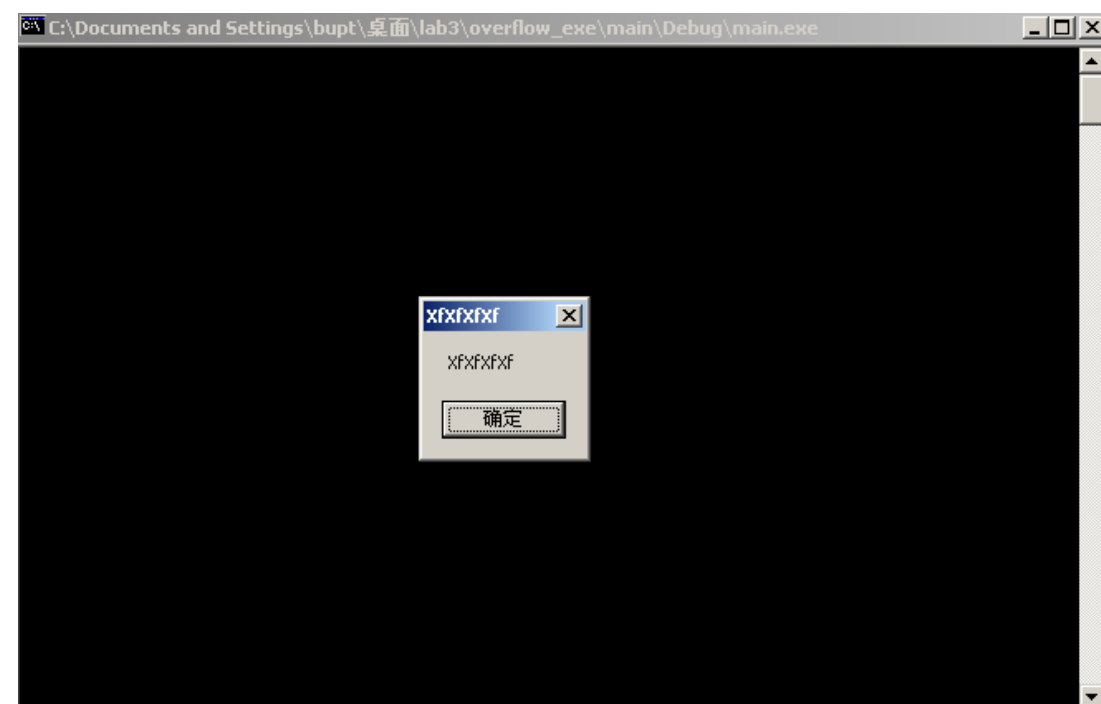


10、修改标题

(1) 修改 password.txt 在 buptbupt 处的操作码，修改如下。



(2) 重新运行 “overflow_exe\main\debug\main.exe”，可以看到弹出窗口。



三、测试结论

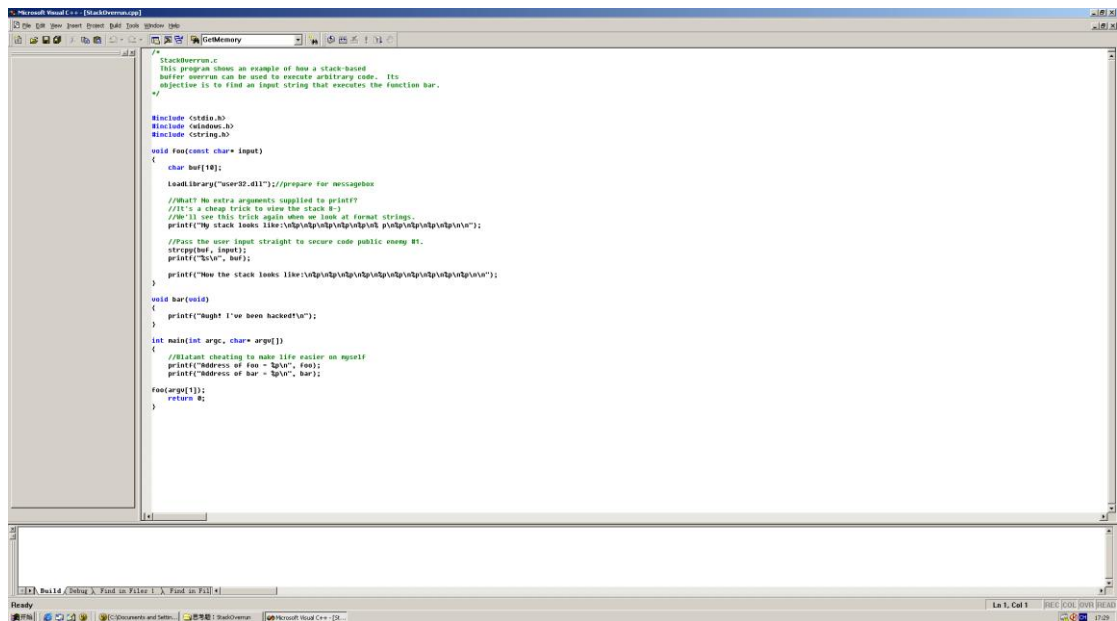
栈溢出对程序运行的安全性有很大的威胁，如果不对栈进行保护，只要系统中存在栈溢出漏洞，即可令攻击者通过非正当方法获取操作权限。因此，需要对栈中保存数据的可执行权限做严格的管理，并且系统要能判断栈溢出从而采取相应的保护措施。

四、思考题

在不修改 StackOverrun 程序源代码的情况下，构造 shellcode，通过 JMP ESP 的方式实现通过记事本打开 shellcode.txt（可使用 CreateProcessA 或 WinExec 等 API）。

1、研究代码

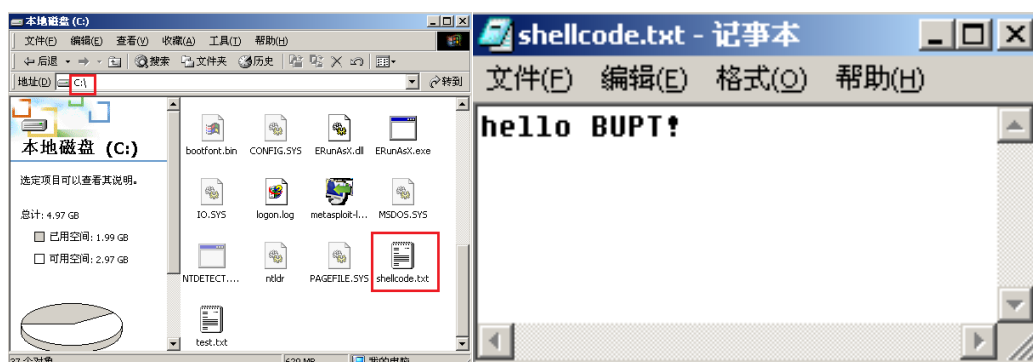
打开 “lab3\思考题：StackOverrun\StackOverrun.cpp”。



可以看到 strcpy 将输入的内容复制到栈中，此处会导致栈溢出漏洞。另外 main 函数有两个参数，第一个参数 int argc 为传入的字符串个数，第二个参数是字符串指针数组，用于存放传入的字符串指针。

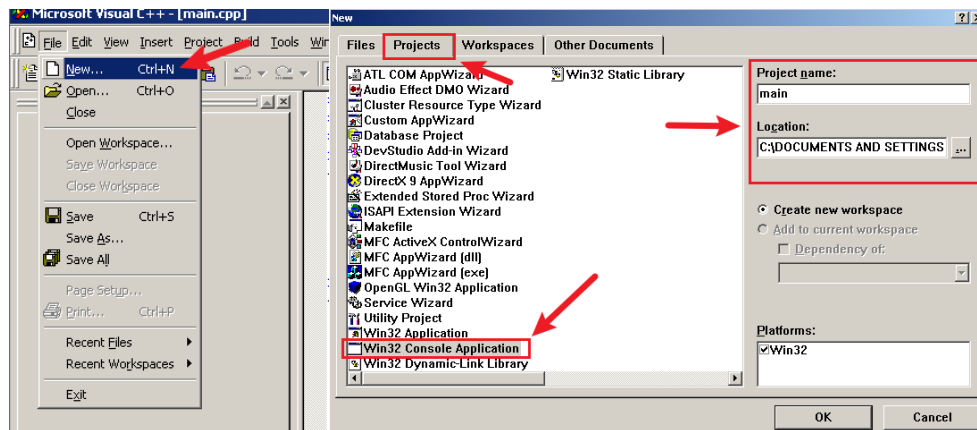
2、编写 shellcode.txt

此处选用 WinExec("cmd.exe notepad.exe C:/shellcode.txt", SW_SHOWNORMAL)，其第一个参数为要打开的程序名称，第二个参数为窗口大小。当调用该命令时还需要加入 cmd 的参数，/k dir 表示执行完 dir 命令后不关闭命令窗口。（注意 shellcode.txt 的文件位置影响到汇编代码与 ollydbg 运行，文件位置与指令指向的位置需要一致）Shellcode.txt 文件位置为 C:/shellcode.txt。

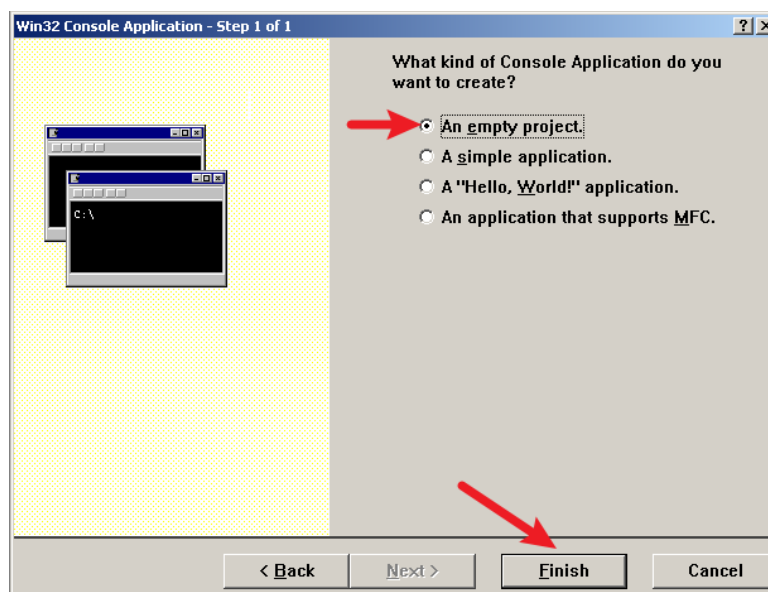


2、生成工程

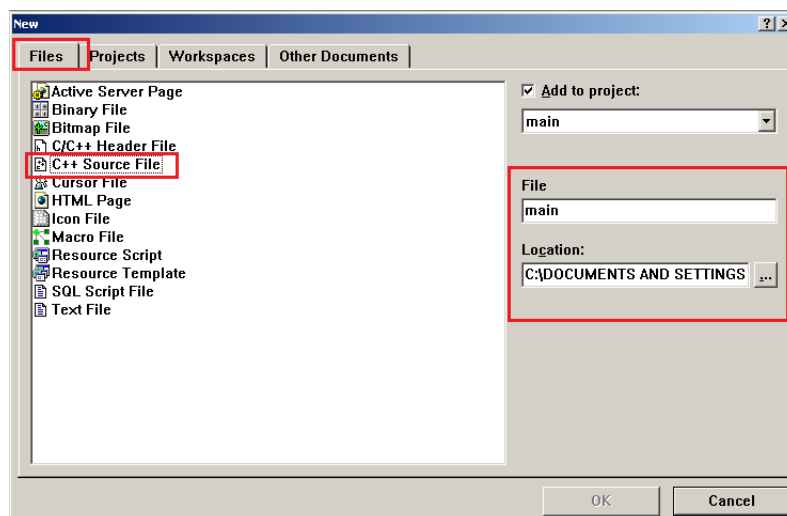
(1) 在 VC 中，点击 File->new->Projects->Win32 Console Application，右上角设置好 project 的名称和存放路径。



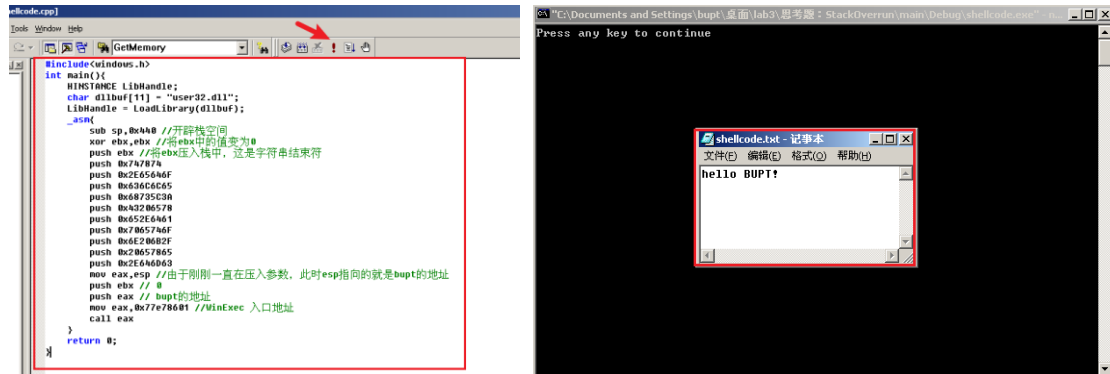
(2) 点击 OK->An empty project->Finish。



(3) 之后再 new->Files->C++ Source File, 右侧设定好添加到的项目(我们刚才新建的项目), 文件名和路径。

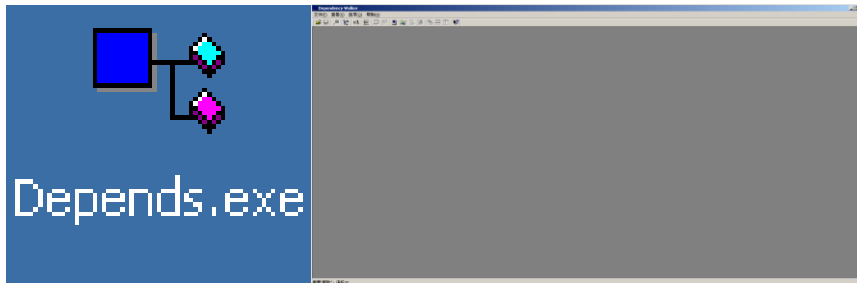


(4) 完成新建文件后，在新建的文件下编写或复制刚才我们看到的 main 程序，然后点击右上的红色感叹号执行。

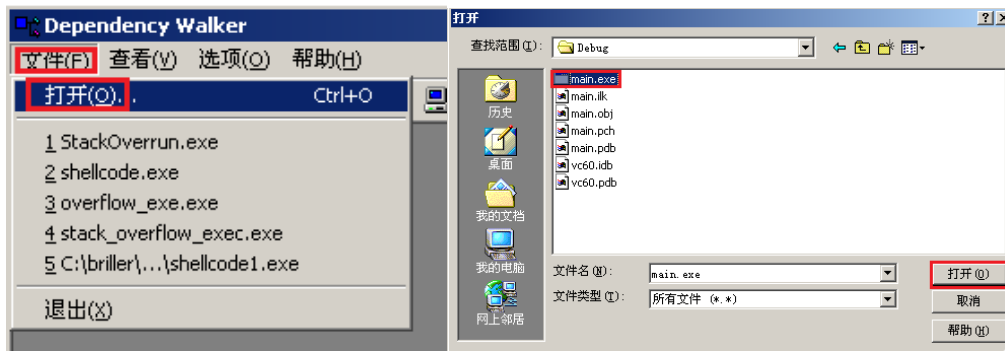


3、获取 WinExec 函数的地址

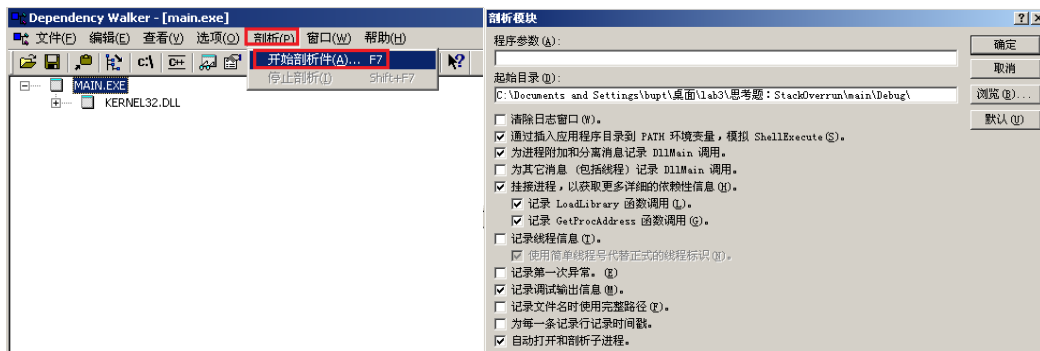
(1) 点击桌面上的 depends.exe 进入程序。



(2) 打开我们生成的 exe 文件。



(3) 点击剖析->开始剖析，参数保持默认即可。



(4) 点击 kernel32.dll，在右侧能找到 WinExec 的函数入口点 0x00018601。

E	序数 ^	提示	函数	入口点
C	766 (0x02FE)	765 (0x02FD)	WaitForSingleObject	0x00008778
C	767 (0x02FF)	766 (0x02FE)	WaitForSingleObjectEx	0x0001E891
C	768 (0x0300)	767 (0x02FF)	WaitNamedPipeA	0x00035A81
C	769 (0x0301)	768 (0x0300)	WaitNamedPipeW	0x00021A47
C	770 (0x0302)	769 (0x0301)	WideCharToMultiByte	0x0000AFD8
C	771 (0x0303)	770 (0x0302)	WinExec	0x00018601
C	772 (0x0304)	771 (0x0303)	WriteConsoleA	0x00014BD4
C	773 (0x0305)	772 (0x0304)	WriteConsoleInputA	0x00045F01
C	774 (0x0306)	773 (0x0305)	WriteConsoleInputVDMW	0x00045C83
C	775 (0x0307)	774 (0x0306)	WriteConsoleInputVDMW	0x00045C9F

(5) 在下面的窗口中能够查到 kernel32.dll 的实际基址 0x77E60000。

模块	文件时间戳	连接时间戳	文件大小	属性	连接名称	校验和	CPU	子系统	语言	版本基址	实际基址	虚拟大小	数据目录	文件版本	产品版本	编译版本	连接名称	CPU	子系统
kernel32.dll	2000-01-10 12:00	1999-12-11 1:14	865,944	A	0x0000CF37	0x0000CF37	x86	Console	DBG	0x77E60000	0x77E60000	0x00006000	3	5.0.2191.1	5.0.2191.1	5.0	5.12	5.0	4.0
USER32.dll	2000-01-10 12:00	1999-12-11 1:14	405,144	A	0x0006AF3F	0x0006AF3F	x86	GUI	DBG	0x77F00000	0x77F00000	0x00064000	4	5.0.2180.1	5.0.2180.1	5.0	5.12	5.0	4.0

(6) 将两个地址相加即可得到 WinExec 的入口地址：

$$0x0018601 + 0x77E60000 = 0x77E78601.$$

4、编写 shellcode，并编译生成 exe 文件。

```

#include<windows.h>
int main(){
    HINSTANCE LibHandle;
    char dllbuf[11] = "user32.dll";
    LibHandle = LoadLibrary(dllbuf);
    _asm{
        sub sp,0x440 //开辟栈空间
        xor ebx,ebx //将ebx中的值变为0
        push ebx //将ebx压入栈中，这是字符串结束符
        push 0x7478742E
        push 0x72696420
        push 0x3E207269
        push 0x6420682F
        push 0x20657865
        push 0x2E646063
        mov eax,esp //由于刚刚一直在压入参数，此时esp指向的就是bupt的地址
        push ebx // 0
        push eax // bupt的地址
        mov eax,0x77E78601 //WinExec 入口地址
        call eax
    }
    return 0;
}

```

5、用 ollydbg 打开刚刚生成的 shellcode.exe 文件，找到对应的汇编语句（XOR EBX,EBX 开始，CALL EAX 结束），并右键复制到文件获取机器码。

```

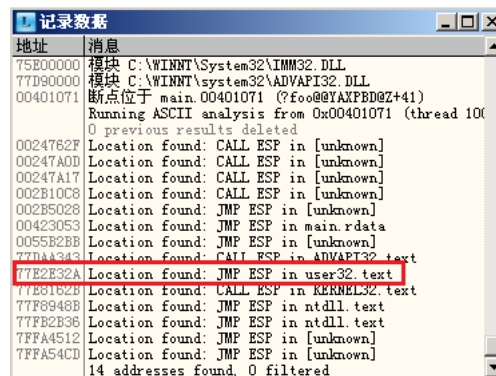
00401051 51          PUSH ECX
00401052 FF15 ACA14200 CALL DWORD PTR DS:[<kernel32.LoadLibraryA>]
00401058 3BF4        CMP ESI,ESP
0040105A E8 81000000 CALL shellcod_chkresp
0040105F 6945 FC     MOV WORD PTR SS:[EBP-4],EAX
00401062 66:81EC 4004 SUB SP,440
00401067 33DB        XOR EBX,EBX
00401069 53          PUSH EBX
0040106A 68 2E747874 PUSH 7478742E
0040106F 68 20646972 PUSH 72696420
00401074 68 6972203E PUSH 3E207269
00401079 68 2F6B2064 PUSH 6420682F
0040107E 68 65786520 PUSH 20657865
00401083 68 636D642E PUSH 2E646063
00401088 8BC4        MOV EAX,ESP
0040108A 53          PUSH EBX
0040108B 50          PUSH EAX
0040108C B8 0186E777 MOV EAX, <kernel32.WinExec>
00401091 FFD0        CALL EAX
00401093 33C0        XOR EAX,EAX
00401095 5F          POP EDI
00401096 5E          POP ESI
00401097 5B          POP EBX
00401098 83C4 50     ADD ESP,50
0040109B 3BEC        CMP EBX,ESP
0040109D E8 3E000000 CALL shellcod_chkresp
004010A2 8BFF        MOV EBP,EBP

```

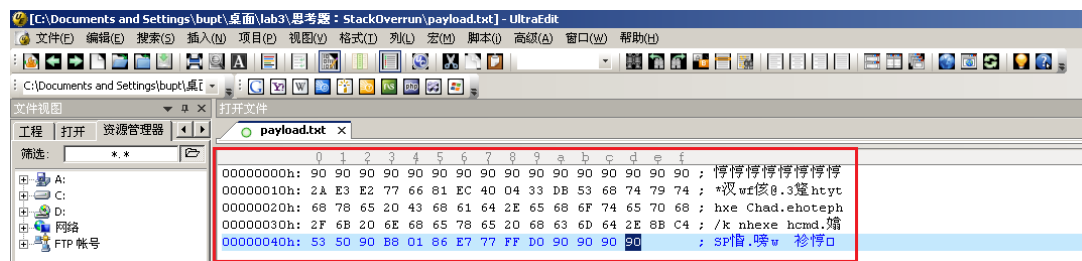

8、搜索完毕后点击日志查看(L 图标按钮)。



9、可以看到一条在 user32.text 中的 JMP ESP 指令的地址为 0x77E2E32A。



10、根据以上信息构造 payload。



11、用 ollydbg 打开 “lab3\思考题：StackOverrun\ main\debug\main.exe”，调试->参数，将前面构造的 payload 作为参数输入。

